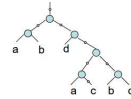


AIG

And-Inverter Graph

ab	00	01	11	10
cd	0	0	1	0
00	0	0	1	0
01	0	0	1	1
11	0	1	1	0
10	0	0	1	0

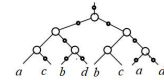
$$(a, b, c, d) = ab + d(ac' + bc)$$



6 nodes
4 levels

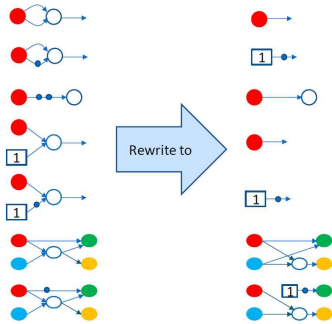
ab	00	01	11	10
cd	0	0	1	0
00	0	0	1	0
01	0	0	1	1
11	0	1	1	0
10	0	0	1	0

$$F(a, b, c, d) = \overline{a}\overline{c}bd + bc\overline{a}d$$



7 nodes
3 levels

Simplifying AIGs: Few Rewrite Rules

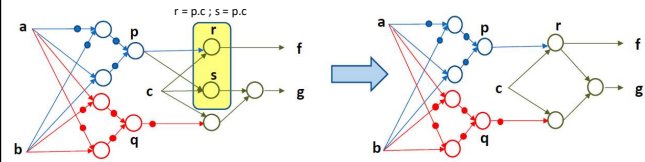


AIG based tools
have tens of such
rewrite rules

Peephole
optimization
restricted to AIGs
of ≈ 4 inputs

Empirically, very
effective in
reducing AIG
sizes

Simplifying AIGs: Structural Hashing



- Before creating a new node with inputs (i_1, i_2) , check if hash table has entry with key (i_1, i_2) or (i_2, i_1)
 - Alternatively, impose ordering on nodes, check for $(\min(i_1, i_2), \max(i_1, i_2))$
- If yes, use value from hash table
- Else, create new node with inputs (i_1, i_2) and make a new entry in hash table

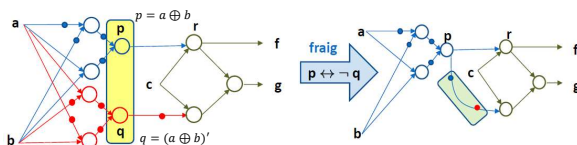
```

Algorithm create_and2(p1, p2) {
  /* constant folding */
  if (p1 == CONST_0) return CONST_0;
  if (p2 == CONST_0) return CONST_0;
  if (p1 == CONST_1) return p2;
  if (p2 == CONST_1) return p1;
  if (p1 == p2) return p1;
  if (p1 == "p2") return CONST_0;
  /* rank order inputs to catch commutativity */
  if (rank(p1) > rank(p2)) swap(p1, p2);
  /* check for isomorphic entry in hash table */
  if ((p = hash_lookup(p1, p2)) == NULL)
    p = new_and_node(p1, p2);
  return p;
}

```

Beyond Strash and Rewrite

Even after using strash and rewrite, AIG may contain nodes representing the same or complemented function



Strash does not help here!

Solution: Check if $(p \leftrightarrow q)$ or $(p \leftrightarrow q')$ is a tautology (SAT solving!)
If yes, merge nodes & use bubbles appropriately on outgoing edges

Simplifying AIGs: FRAIG

Functionally Reduced AIG (FRAIG)

No two nodes represent the same or complemented function (Similar to ROBDD)

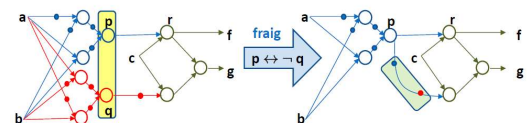
FRAIG Overview

For every pair (u, v) of nodes (including constant node 1)

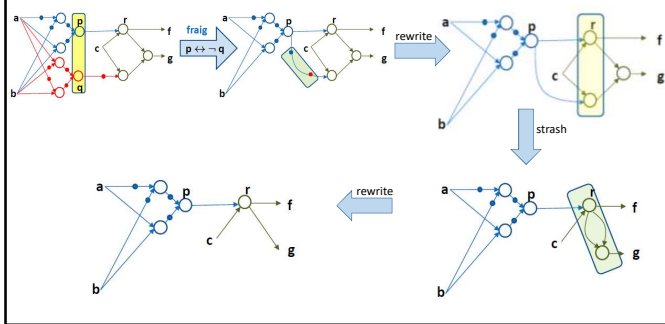
If $(u \leftrightarrow v)$ is a tautology, merge nodes u and v in AIG

If $(u \leftrightarrow v')$ is a tautology, merge nodes u and v in AIG, and use bubbles appropriately on outgoing edges

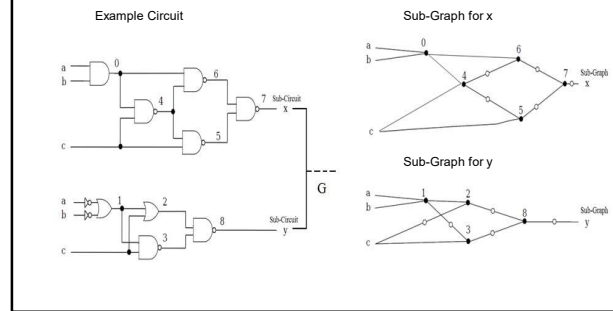
Use simplification rules and structural hashing all throughout



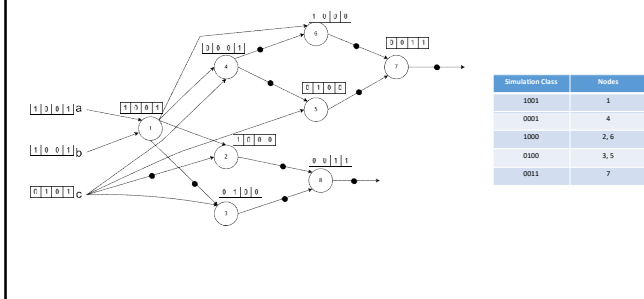
Simplifying AIGs: fraig, strash, rewrite



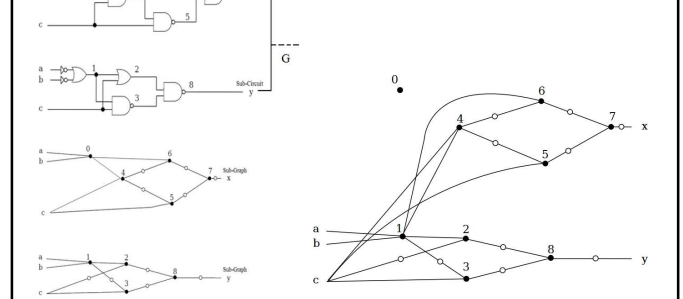
FRAIG Construction



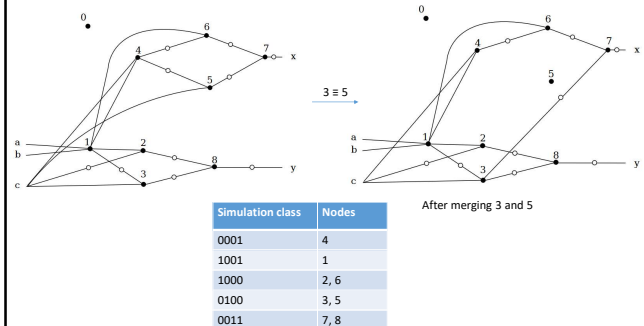
FRAIG Construction: Random Simulation



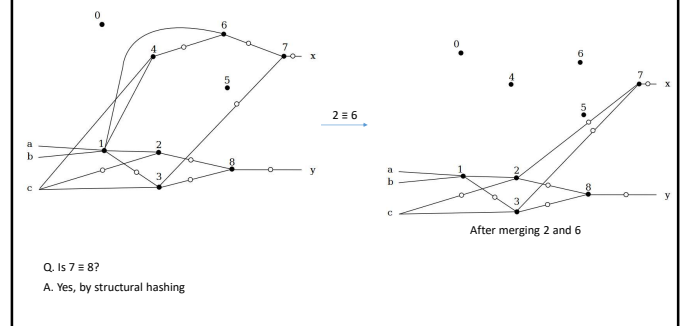
FRAIG Construction



FRAIG Construction: Equivalence Checking



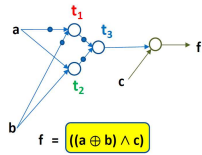
FRAIG Construction: Equivalence Checking



AIG to CNF: Tseitin Encoding

Given Boolean circuit (or AIG) for f

- Associate new variable with output of each gate
- For each Boolean gate, generate formula expressing output in terms of inputs
- Convert each formula generated above to CNF
 - Boolean gate with fixed # inputs $\rightarrow$ CNF formula for gate is of fixed size
- Conjoin CNF formula for each gate and output variable of circuit



$$t_1 \leftrightarrow (a \wedge \neg b)$$

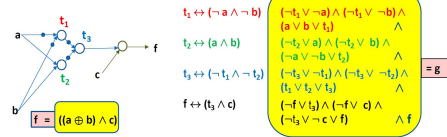
$$t_2 \leftrightarrow (a \wedge b)$$

$$t_3 \leftrightarrow (\neg t_1 \wedge \neg t_2)$$

$$f \leftrightarrow (t_3 \wedge c)$$

$$\begin{aligned} &(\neg t_1 \vee \neg a) \wedge (\neg t_1 \vee \neg b) \wedge \\ &(a \vee b \vee t_1) \wedge \\ &(\neg t_2 \vee a) \wedge (\neg t_2 \vee b) \wedge \\ &(\neg a \vee \neg b \vee t_2) \wedge \\ &(\neg t_3 \vee \neg t_1) \wedge (\neg t_3 \vee \neg t_2) \wedge \\ &(t_1 \vee t_2 \vee t_3) \wedge \\ &(\neg f \vee t_3) \wedge (\neg f \vee c) \wedge \\ &(\neg t_3 \vee \neg c \vee f) \wedge f \end{aligned}$$

AIG to CNF: Tseitin Encoding



$$t_1 \leftrightarrow (a \wedge \neg b)$$

$$t_2 \leftrightarrow (a \wedge b)$$

$$t_3 \leftrightarrow (\neg t_1 \wedge \neg t_2)$$

$$f \leftrightarrow (t_3 \wedge c)$$

$$\begin{aligned} &(\neg t_1 \vee \neg a) \wedge (\neg t_1 \vee \neg b) \wedge \\ &(a \vee b \vee t_1) \wedge \\ &(\neg t_2 \vee a) \wedge (\neg t_2 \vee b) \wedge \\ &(\neg a \vee \neg b \vee t_2) \wedge \\ &(\neg t_3 \vee \neg t_1) \wedge (\neg t_3 \vee \neg t_2) \wedge \\ &(t_1 \vee t_2 \vee t_3) \wedge \\ &(\neg f \vee t_3) \wedge (\neg f \vee c) \wedge \\ &(\neg t_3 \vee \neg c \vee f) \wedge f \end{aligned}$$

Bijection between SAT assignments of f and g

For every SAT assignment of f

- Evaluate all gate outputs for the given assignment of inputs

- $a = 1, b = 0, c = 1$ gives $a = 1, b = 0, c = 1, t_1 = 0, t_2 = 0, t_3 = 1, f = 1$ (SAT assignment of g)

For every SAT assignment of g

- Project assignment on the variables of f

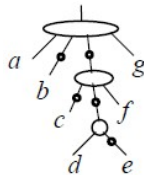
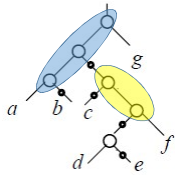
- $a = 1, b = 0, c = 1, t_1 = 0, t_2 = 0, t_3 = 1, f = 1$ gives $a = 1, b = 0, c = 1$ (SAT assignment of f)

AIG Balancing

And balancing: performed in two steps – covering and tree balancing

Covering: identifies large multi-input ANDs in the AIG by grouping together two-input ANDs

- have **no complemented attributes** in between and **no external fanout**, except possibly at the root node of each multi-input AND.

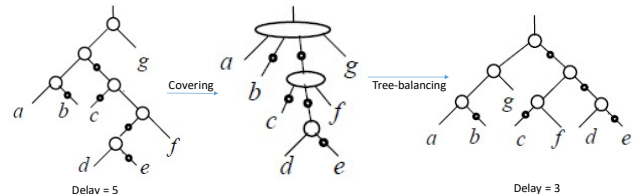


AIG Balancing

And balancing: performed in two steps – covering and tree balancing

Tree balancing: decomposes each multi-input AND into two-input ANDs while trying to reduce the total number of AIG levels.

- tree-balancing step is not unique and depends on the grouping of the inputs with equal delay.



Implemented in ABC as command **balance**

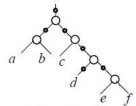
AIG Balancing

SOP balancing of a small* AIG:

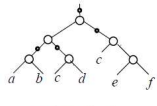
- Convert into an SOP.
- AND-balancing can be applied to each product and the sum.
- The products and the sum are treated as multi-input ANDs and decomposed to minimize the delay of the output node

$$F = ab + c(d + ef)$$

$$F = ab + cd + cef$$



Delay: 4 levels



Delay: 3 levels

Q. Can AND balancing reduce the delay for this example?

SOP-balancing can reduce delay when AND-balancing cannot

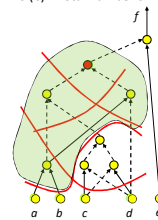
Q. How SOP balancing can be performed on a large AIG?

* an AIG is considered *small* if it depends on roughly 10 or less inputs

Cut of an AIG Node

Definition: A cut C of node n is a set of nodes in the network (aka *leaves*), such that each path from a primary input to n passes through at least one leaf.

- Node n is called root of the cut.
- The cut size is the number of its leaves. K -feasible cut has the number of leaves $\leq K$
- A cut C is said to be **dominated** if there is another cut C' of the same node such that $C' \subseteq C$.
- $\text{Vol}(C)$ = Total number of nodes encountered on all paths between node n and the cut leaves.



Q. How to find all cuts of a node?

Enumerating of all K -feasible cuts of a node

Let A and B be two sets of cuts.

$$A \bowtie B = \{u \cup v \mid u \in A, v \in B, |u \cup v| \leq K\}$$

Let $\Phi(n)$ denote the set of K feasible cuts of node n .

If n is a node with n_1 and n_2 as fanins, then,

$$\Phi(n) = \begin{cases} \{n\}, & n \in PI \\ \{ \{n\} \cup \Phi(n_1) \bowtie \Phi(n_2) \}, & \text{Otherwise} \end{cases}$$

Cut Enumeration

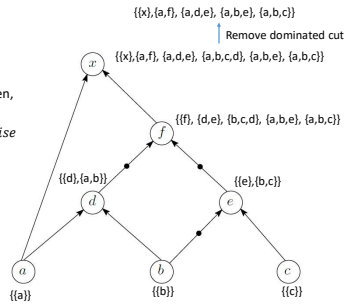
Enumerating of all K -feasible cuts of a node

Let A and B be two sets of cuts.

$$A \bowtie B = \{u \cup v \mid u \in A, v \in B, |u \cup v| \leq K\}$$

$\Phi(n)$: the set of K feasible cuts of node n .
If n is a node with n_1 and n_2 as fanins, then,

$$\Phi(n) = \begin{cases} \{n\}, & n \in PI \\ \{\{n\}\} \cup \{\Phi(n_1) \bowtie \Phi(n_2)\}, & \text{Otherwise} \end{cases}$$



SOP Balancing of a Large AIG

- It is impossible to apply SOP balancing to a large AIG.
- Break large AIG into parts.
- SOP-balancing for each part, and if the delay is improved, locally update the large AIG with the structure derived by SOP-balancing.

```

subject_graph performSopBalancing (
  subject_graph S; // S is an And-Inverter Graph
  int K; // K is the cut size
  int C; // C is the number of cuts at each node
)
{
  for each node n in S, in a topological order {
    compute C structural K-input cuts of n;
    for each cut {
      compute truth table;
      compute irredundant SOP;
      perform delay-optimal balancing of the SOP;
      if (the cut has smaller AIG level than the best cut)
        save the cut as the best cut;
    }
    if (root node AIG level is reduced using the best cut)
      update AIG structure;
  }
  return S;
}

```

Classification of n -input Boolean Function

NPN equivalence: Two Boolean functions, F and G , belong to the same NPN-class (aka NPN equivalent) if $\bar{F}$ can be derived from $\bar{G}$ by negating (N) and permuting (P) inputs and negating (N) the output.

Q: check for NPN equivalence $ab+c$ and $a'c'+b'c'$

Ans: $[c'(a'+b)'] \equiv c+ab$

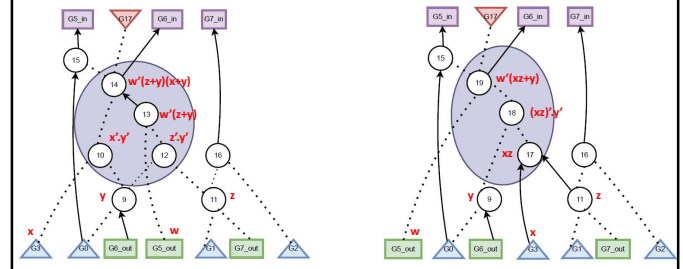
What is the complexity of the exhaustive method for NPN Boolean matching? $O(n! 2^{n+1})$

NPN classes of 2-variable Boolean functions	# of inputs	Functions	NPN classes
0, 1	1	4	2
a, a', b, b'	2	16	4
$a+b, a'+b, a+b', a'+b', ab, a'b, ab', a'b'$	3	256	14
$a'b+ab', a'b'+ab$	4	65536	222

# of inputs	Functions	NPN classes
1	4	2
2	16	4
3	256	14
4	65536	222

AIG Rewriting

- Rewriting** is a fast greedy algorithm
- Minimizing the AIG size by iteratively selecting AIG subgraphs rooted at a node and replacing them with smaller pre-computed subgraphs
- Preserving the functionality of the root node.



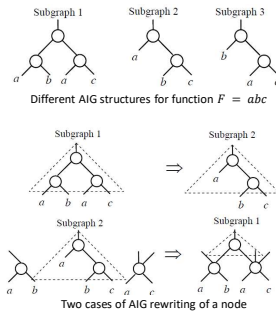
AIG Rewriting

Rewriting: network AIG, hash table PrecomputedStructures, bool UseZeroCost

```

{
  for each node N in the AIG in the topological order {
    for each 4-input cut C of node N computed using cut enumeration {
      F = Boolean function of N in terms of the leaves of C
      PossibleStructures = HashTableLookup( PrecomputedStructures, F );
      // find the best logic structure for rewriting
      BestS = NULL; BestGain = -1;
      for each structure S in PossibleStructures {
        NodesSaved = DifferenceNode( AIG, N );
        NodesAdded = ReferenceNode( AIG, S );
        Gain = NodesSaved - NodesAdded;
        Difference( AIG, S ); Reference( AIG, N );
        if ( Gain > 0 || ( Gain == 0 && UseZeroCost ) )
          if ( ( BestS == NULL || BestGain < Gain ) )
            BestS = S; BestGain = Gain;
      }
      if ( BestS == NULL ) continue;
      // use the best logic structure to update the netlist
      NodesSaved = DifferenceNode( AIG, N );
      NodesAdded = ReferenceNode( AIG, BestS );
      assert( BestGain == NodesSaved - NodesAdded );
    }
  }
}

```

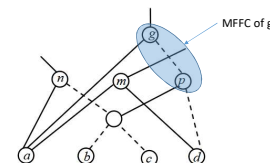


Two cases of AIG rewriting of a node

Resubstitution

- Resubstitution expresses one function in terms of other functions.
- Given $f(x)$ and $g_1(x), g_2(x), \dots, g_k(x)$, is it possible to express f in terms of a subset of g_i 's?
- If so, what is function $f(g)$?

A maximum fanout free cone (MFFC) of node n is a subset of the fanin cone, such that every path from a node in the subset to the POs passes through n . Informally, MFFC of a node contains all the logic used exclusively by the node.



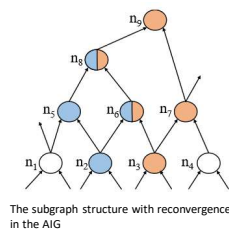
Reconvergence

Reconvergence occurs when paths originating at a node's output meet again before reaching the POs.

Q? Can you find a reconvergence in the AIG:

Ans: n_2 's fanout meets at n_8 before reaching PO. Also for n_3 .

- Reconvergence is inevitable due to logic sharing in multi-level logic networks.
- Excessive reconvergence is often redundant -- opportunities for optimization.



Reconvergence-driven Cut

Reconvergence occurs when paths originating at a node's output meet again before reaching the POs.

```

nodeset ReconvergenceDrivenCut( node N, int CutSizeLimit )
{
  nodeset Leaves = { N };
  nodeset Visited = { N };
  ConstructCut_rec( Leaves, Visited, CutSizeLimit );
  return Leaves;
}

ConstructCut_rec( nodeset Leaves, nodeset Visited, int CutSizeLimit )
{
  if ( Leaves contain only PI nodes )
    return;
  M = non-PI node in Leaves with the minimum LeafCost;
  if ( (Leaves) + LeafCost( M, Visited ) > CutSizeLimit )
    return;
  Leaves = Leaves - fanins( M );
  Visited = Visited ∪ fanins( M );
  ConstructCut_rec( Leaves, Visited, CutSizeLimit );
}

int LeafCost( node M, nodeset Visited )
{
  int Cost = -1;
  for each fanin F of node M
    if F ∉ Visited
      Cost = Cost + 1
  return Cost
}

```

Leaves = {n9} Visited = {n9}
 Leaves = {n7, n8} Visited = {n9, n7, n8} LeafCost(n7) = LeafCost(n8) = 1
 Leaves = {n8, n3, n4} Visited = {n9, n7, n8, n3, n4}
 Leaves = {n5, n6, n3, n4} Visited = {n9, n7, n8, n3, n4, n5, n6}
 LeafCost(n6) = 0 and LeafCost of n5, n3, n4 are 1
 Leaves = {n5, n2, n3, n4} Visited = {n9, n7, n8, n3, n4, n5, n6, n2}

Note:

- ConstructCut_rec** expands the cut by incrementally adding one node. Greedily minimize the #cut leaves in each iteration
- LeafCost(N) ∈ {-1, 0, 1}**. LeafCost(N) = -1 if N and both of its fanins are in the cut Leaves, 0 or 1
- ConstructCut_rec** prefers the nodes with the cost -1, and 0 (such nodes are cut leaves having reconvergent path inside the cut)

Resubstitution in ABC

Resubstitution(network AIG, int CutSizeLimit, int DivisorLimit, bool UseZeroCost)

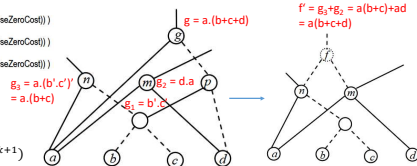
```

{
  for each node n in the AIG in the topological order {
    int MfcSize = | MFFC( n ) |; assert( MfcSize >= 1 );
    nodeset C = ReconvergenceDrivenCut( n, CutSizeLimit );
    nodeset D = CollectNodesTFOChanged( C, Level(n), DivisorLimit );
    ComputeFunctions( D, n );
    nodeset R = TryResubstitution( n, D );
  }
  if ( R == NULL && (MfcSize > 1) && (MfcSize == 1 && UseZeroCost) )
    R = TryResubstitution( n, D );
  if ( R == NULL && (MfcSize > 2) && (MfcSize == 2 && UseZeroCost) )
    R = TryResubstitution( n, D );
  if ( R == NULL && (MfcSize > 3) && (MfcSize == 3 && UseZeroCost) )
    R = TryResubstitution( n, D );
  if ( R != NULL )
    UpdateNetwork( AIG, n, R );
}

```

Cut with 12-16 leaves are used -- resub can be performed with TT and exhaustive simulation

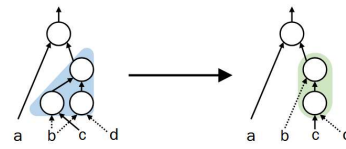
- The collected node's level does not exceed the level of node n
- Leaves of the cut are collected, but nodes in the MFFC are skipped
- #collected nodes (divisors) ≤ 150



Note: The level of a node is the length of the longest path from any PI to the node. The node itself is counted towards the path lengths but PIs are not. Inverters are not counted in the path length.

Refactoring

- Optimizes the area of AIG
- Resynthesize large logic cones
 - By algebraic factoring from collapsed SOPs
 - Using one cut per node to reduce time complexity
- Replace the original cone if the resynthesized cone has positive gain



Refactoring in Parallel

Stage 1: identify and collapse logic cones (subgraphs) that are

- (1) disjoint from each other, and
- (2) cover all the logic (non-PI nodes) in the AIG

Stage 2: resynthesize all local functions in parallel

Stage 3: replace the original cones by the resynthesized cones in parallel

